

Master's Thesis

Toward 200 Gbps Content Servers
Via Utilizing SmartNICs

Dong U Kim

Department of Computer Science and Engineering

Ulsan National Institute of Science and Technology

2024

Toward 200 Gbps Content Servers Via Utilizing SmartNICs

Dong U Kim

Department of Computer Science and Engineering

Ulsan National Institute of Science and Technology

Abstract

We present FlexServer, the first high-performance content server that supports a line rate of 200 Gbps NICs. We use a SmartNIC to offload disk read and packet transfer operations. To reduce the overhead of disk read operations, we utilize the up-to-date asynchronous I/O mechanism, *io_uring*, and hide the latency of disk read operations. To reduce the inefficiency of duplicate transmission of popular content, we introduce a caching mechanism both at the server and the SmartNIC. We also develop a mechanism to allocate the workload to the host and SmartNIC dynamically. We deploy a dual buffer-based consistency management algorithm to the control plane for complete consistency with TCP. We implement FlexServer using the mTCP user-level TCP stack and an NVIDIA Bluefield 2 SmartNIC with a 200 Gbps Ethernet port. Via evaluations, we show that FlexServer can achieve the line rate of 200 Gbps with only a single host CPU core in typical workload scenarios.

Contents

I	Introduction	1
II	Background & Motivation	3
	2.1 Limitations of Existing High Performance Content Servers	3
	2.2 Recent Advancement in Disk I/O Mechanisms	3
	2.3 Streamlining the Data Plane by Utilizing SmartNICs	3
III	Design of FlexServer	5
	3.1 Design Principles	5
	3.2 Efficient division of control and data plane over the host and SmartNIC	6
	3.3 Hiding the disk read latency using asynchronous disk I/O	7
	3.4 Hierarchical caching using the host and SmartNIC memory	9
	3.5 Dual Buffer-based Consistency Management in the Control Plane	10
	3.6 Intelligent load balancing between the host and SmartNIC	13
IV	Implementation	15
	4.1 Host	15
	4.2 SmartNIC	15
V	Evaluation	16
	5.1 Experiment Setup	16

5.2	Experimental Results	16
VI	Related Work	21
VII	Conclusion	22
	References	23
	Acknowledgements	25

List of Figures

1	Architecture of FlexServer.	6
2	Operation of asynchronous file read using <i>io_uring</i>	8
3	Structure of the fields in the intermediate packet.	11
4	Example of the host TCP operations utilizing dual buffers.	12
5	Throughput of FlexServer, Linux with <i>sendfile()</i> , and mTCP when disk reads are performed on the host while varying the number of CPU cores.	16
6	Flame graph when running FlexServer with 1 CPU core.	17
7	Flame graph when running Linux with 1 CPU core.	18
8	Flame graph when running mTCP with 1 CPU core.	19
9	Throughput by SmartNIC cache and file reads when varying the number of cache hit flows and file read flows.	19

I Introduction

According to Sandvine’s 2023 Global Internet Phenomena Report [1], Internet video traffic accounts for 65% of all Internet traffic in 2022. The report also reveals that marketplace, gaming, social networking, cloud, web browsing, and file-sharing applications followed video applications by total volume. Lots of these application services are based on content servers that provide the delivery of a large volume of static content to users. Establishing an efficient content delivery system to improve user service quality and reduce service costs is crucial for service providers serving a large number of users.

One of the main challenges for implementing high-performance content servers is the increasing gap between the growth speeds of CPUs and I/O devices such as NVMe SSD and NICs. The content server should bring the content into the main memory before delivering the content to users, and during this procedure, lots of CPU cycles are consumed, and the memory bandwidth easily becomes the performance bottleneck. There have been different research works that try to improve the performance of content servers by reducing this inefficiency. Atlas server [2] avoids using the inefficient disk buffer cache, and by using DDIO technology [3], the video chunk is processed in the Last Level Cache (LLC), by DMAing the chunk from disks into the LLC, encrypting in place, packetizing, and DMAing to NIC without accessing the RAM. IO-TCP [4] is a content server architecture that offloads the data-plane operations such as disk I/O and packet transfer into SmartNICs while the control plane operations are executed on the host CPU. Even though these solutions mitigate the inefficiency of the content servers and improved the throughput performance, no solution provided the line-rate performance of recent high-performance NICs, e.g., 200 Gbps, with TCP support.

We identify three challenges in improving the throughput performance of content servers to support 200 Gbps NICs. First, despite the efforts of previous works, the file read operations still incur a large overhead. For example, due to a large number of disk I/O operations, the memory bandwidth in SmartNIC becomes the performance bottleneck of IO-TCP. To mitigate the disk I/O overhead, we note the recent advancement in the Linux I/O abstract, i.e., *io_uring* [5]. *io_uring* is a Linux’s system call interface for asynchronous I/O operations of storage devices, introduced in Linux kernel version 5.1. It offers ring buffers for the main interface of communication between the kernel and user-space and has been proven to provide high performance and efficient CPU utilizations [6]. Second, although the I/O operation offloading approach by the state-of-the-art solution, IO-TCP, significantly increases the throughput performance, it does not take the best advantage of both the CPU and SmartNIC resources by eliminating the caching mechanism. The operating system’s disk buffer cache can be inefficient, but in many service scenarios, we can eliminate the duplicate I/O operations for popular content by utilizing the custom memory cache in the host and SmartNIC. For optimal performance, we also need dynamic load balancing between the host and SmartNIC, depending on the variation of the request amount and content popularity distribution. Third, we need to maintain consistency with TCP even when multiple objects for a single flow are offloaded to SmartNIC for disk read or cache-based packet transfer. IO-TCP does not support offloading multiple objects simultaneously, so it can not support HTTP 2 multiplexing.

To overcome these challenges, We present FlexServer, a high-performance content server that can

provide a line-rate throughput of 200 Gbps NICs for the first time. We use a SmartNIC to offload disk read and packet transfer operations as in IO-TCP. In addition, to mitigate the overhead of file read operations, we utilize the up-to-date asynchronous I/O mechanism of *io_uring*. Using *io_uring*, we hide the latency of disk read operations and use the CPU cycles more efficiently. To mitigate the inefficiency of duplicate transmission of popular content, we introduce caching both at the server and the SmartNIC. We also develop a mechanism to dynamically allocate the workload into the host and SmartNIC with the aim of fully utilizing the SmartNIC resources preferentially by processing as many requests as possible in SmartNIC and then using the host resources for processing the remaining requests. For complete consistency with TCP when offloading to SmartNIC, we develop a dual buffer-based consistency management algorithm in the control plane.

We implement FlexServer using a NVIDIA Bluefield 2 SmartNIC with a 200 Gbps Ethernet port. We utilize mTCP, a user-level TCP, for the host’s TCP stack and implement a simple API so that applications can easily utilize FlexServer’s capabilities with minor code changes. Through extensive evaluations, we show that the proposed system can achieve significantly higher performance than Linux using *sendfile()*, mTCP, and IO-TCP. Even using only two CPU cores at the host without using a SmartNIC, FlexServer with *io_uring*-based read operations exceeds the maximum throughput of IO-TCP (about 80 Gbps). By using 8 CPU cores and without SmartNIC offloading, FlexServer can achieve almost the line rate of 200 Gbps NIC. When utilizing only SmartNIC, we verified that depending on the cache hit ratios, we can achieve from 36 Gbps to 75 Gbps. We expect that due to the high bandwidth of SmartNIC memory and the increased number of ARM cores in more recent versions of SmartNICs (e.g., 90 GB/s and up to 16 in Bluefield 3 DPUs), FlexServer can support a 400 Gbps line rate with minor adaptations.

The main contributions of this work are as follows. (1) We identify the performance enhancement opportunities of existing content servers from up-to-date asynchronous I/O, custom caching, and load-balancing mechanisms. (2) We present FlexServer, a high-performance content server architecture that can fully utilize the SmartNIC and host resources by allocating disk read operations and cache-based transmissions to SmartNIC first and then to the host. (3) We validate that FlexServer can achieve the line rate of 200 Gbps in typical workload scenarios for the first time.

II Background & Motivation

2.1 Limitations of Existing High Performance Content Servers

There have been different research works for improving the performance of content servers. IO-TCP [4], the state-of-the-art solution, offloads the data-plane operations, including the disk I/O, encryption, and network I/O, into SmartNICs, and the control plane operations are executed on the host CPU. IO-TCP host stack sends to a SmartNIC a TCP packet that carries 4 tuples, sequence/ACK numbers, and the payload contains SEND command with file ID, start offset, and length. With the contained information in the SEND command, SmartNIC reads the file, replaces the payload with the read data, and transfers the modified packet to NIC hardware. The NIC hardware performs TCP packet segmentation and checksum calculations and sends the packet to the wire. When the NIC reads the file in disks, it uses the POSIX read, but this is not efficient since it consumes significant ARM core cycles and incurs the overhead of interrupt handling and kernel-user space context switching. Also, even if host CPU cores are designated, they are not fully utilized by reading files from disks because I/O operations are statically assigned to SmartNIC. IO-TCP removes the overhead of the operating system's file buffer cache by using direct I/O, but it also eliminates the possibility of using cache for popular content. Due to these limitations, IO-TCP cannot provide the line-rate performance of recent high-performance NICs.

2.2 Recent Advancement in Disk I/O Mechanisms

Linux supports two widely used APIs for file I/O operations: POSIX file I/O APIs for synchronous operations and *libaio* for asynchronous operations. Both incur a high overhead because they use the system calls to interact with the kernel. A more recent advanced API provided by Linux is *io_uring*. *io_uring* provides two queues, submission queues (SQs) and completion queues (CQs), to the application to interact with the kernel for storage operations [7]. Applications submit I/O requests to SQs and get notified of the completions through the CQs. By default, *io_uring* uses *io_uring_enter* system call to notify the kernel of new request submissions and also to retrieve the completion events of the requested I/O operations. To remove these system calls for completion processing, a polling mode by application (completion polling mode) is provided. Submission polling mode is also provided to remove the system calls for request submissions. We utilize *io_uring* to hide the latency of disk read operations at both the host and SmartNIC and improve the performance of these operations by minimizing the consumption of CPU cycles.

2.3 Streamlining the Data Plane by Utilizing SmartNICs

A SmartNIC is a device that adds CPUs, NPU (Network Processing Units), memory, accelerators, FPGAs, etc., to a traditional NIC to run user-programmed tasks, allowing the NIC to offload and perform various jobs otherwise handled by the host. SmartNICs have been used to accelerate different systems, such as distributed applications [8], serverless computing systems [9], key-value stores [10], mobile communication networks [11, 12], and TCP [4, 13, 14].

Utilizing a SmartNIC can accelerate the data plane of the content server by placing popular content in the SmartNIC's memory and utilizing it as a cache, as well as by running disk-read operations on SmartNIC. There are several challenges to using this approach to accelerate the data plane of a content server for the line-rate throughput of 200 Gbps NICs.

- **Limited SmartNIC resources:** SmartNICs are generally equipped with CPUs of low clock speeds and small cache capacities compared to typical CPUs used in hosts. Therefore, the cycle budget of the SmartNIC core is limited, so the processing cycles per packet must be minimized. Also, SmartNICs have a relatively small amount of memory. These characteristics require an optimal selection of objects to offload for cache and file read operations.
- **Synchronization between the host and SmartNIC:** In our system, the processing of requests is split between the host and the SmartNIC, and both can handle them by cache and disk read. Also, flows can send data in which normal data, file data, and cached data are mixed (e.g., HTTP 2 multiplexing). It is important to ensure that TCP maintains the same semantics as if it works as a normal TCP, even in complex situations such as packet losses and retransmissions.
- **Load balancing between the host and SmartNIC:** Since the applications run on the host CPUs and their resource requirements can vary, it is advantageous to assign as much workload as possible to SmartNIC and assign the remaining workload to host CPUs. For this, we need to accurately find the request-handling capabilities of SmartNICs and offload the workload as much as it can handle. The request handling capabilities depend on how many requests will be processed by the cache and how many requests will be processed by disk reads.

III Design of FlexServer

In this Section, we overview how we designed the proposed system to overcome the challenges described in Section II and provide the details of the system architecture and algorithms of FlexServer.

3.1 Design Principles

FlexServer is a novel content server architecture that exploits SmartNICs to accelerate a content server's data plane to maximize the throughput and resource utilization. FlexServer has the following design principles.

- **Efficient division of control and data plane over the host and SmartNIC.** Since the host has high-speed CPUs that are suitable for complex control plane processing, our system runs the control plane on the host, and with the remaining host resources and SmartNIC resources, we run the data plane. We design the SmartNIC data plane as simple, separate, and stateless. This minimizes the load for synchronization between SmartNIC and the host and maximizes the request processing capacity of the SmartNIC.
- **Hiding the disk read latency using asynchronous disk I/O.** Disk read is the main overhead of the content servers, so we target to minimize the resource consumption due to disk reads. For this, we utilize Linux's recent advanced I/O abstract, *io_uring*. We integrate *io_uring* into the host's user-level network stack and also into the SmartNIC's packet processing module. This enables hiding the disk read latency through the asynchronous operations and using the saved CPU cycles for other tasks such as packet processing at the host and SmartNIC and control plane processing at the host.
- **Hierarchical caching using the host and SmartNIC memory.** We remove the disk buffer cache, which incurs a large overhead when new data is read from disks. Instead, we maintain a hierarchical custom memory cache on the host and SmartNIC. We set the SmartNIC cache as high-level cache (L1 cache) and the host cache as low-level cache (L2 cache). The L2 cache is exclusive of the L1 cache, and we place the most popular content on the L1 cache and the remaining popular content on the L2 cache by tracking the popularity of content in real-time.
- **Dual buffer-based consistency management in the control plane.** To maintain the consistency of TCP in a situation where multiple types of content are mixed, i.e., normal data, file data, and cached data, we introduce consistency management utilizing dual buffers on the control plane: general send buffer and meta-data send buffer. Using dual buffers, we can clearly match the TCP sequence numbers into their corresponding data byte of different types. Therefore, we can correctly handle various TCP events, such as packet losses and retransmissions.
- **Intelligent load balancing between the host and SmartNIC.** Since both the host and SmartNIC perform transmission of three types of content, it is not trivial to decide which one should handle

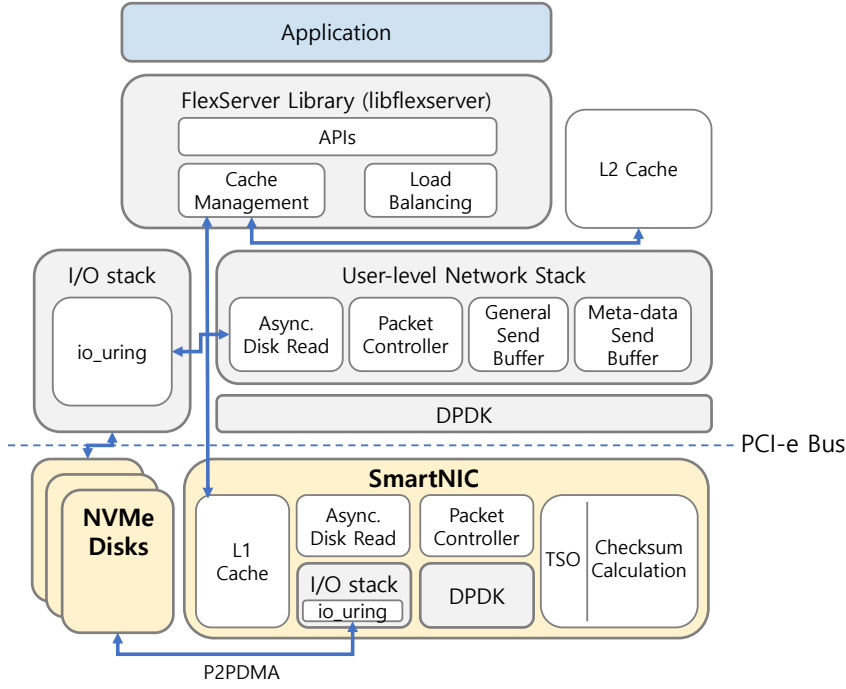


Figure 1: Architecture of FlexServer.

a specific request. Our objective for load balancing is to assign as much workload as possible to SmartNIC and assign the remaining to host CPUs. For this, we devise algorithms for estimating throughputs by cache-hit objects at both L1 and L2 caches and for estimating the available capacity of SmartNIC that can process the requests by file reads. The SmartNIC is assigned the file read workload from the estimation, and the remaining workload is assigned to the host.

3.2 Efficient division of control and data plane over the host and SmartNIC

The architecture of FlexServer is shown in Figure 1. The FlexServer library (*libflexserver*) is located between the applications and I/O stack, network stack. The FlexServer library manages the L1 cache at the host’s memory and the L2 cache at the SmartNIC’s memory and performs load balancing by assigning request processing to the host and SmartNIC. It also provides APIs, including functions that applications can call to allocate and release resources for each library instance, to check whether an object is in the cache or not, and to transmit a file object. The Packet Controller at the host inserts the meta-data for the object to send into a TCP packet without payload if the object is to be sent by the SmartNIC. When the SmartNIC receives the packet, the Packet Controller performs asynchronous disk read through *io_uring* or retrieves the object from L1 Cache, inserts the object in the packet, and sends it out after TSO/checksum calculation. If the object is decided to be sent by the host, the host retrieves the object by asynchronous disk read or cache read, creates the TCP packet, and sends it out directly after TSO/checksum calculation. The General Send Buffer is used for storing data that is not a file object, e.g., HTTP headers. The Meta-data Send Buffer is for storing the TCP sequence number information for the file object.

The SmartNIC performs its role by carrying out commands from the host. The commands from the host to the SmartNIC data plane are OFFLOAD, EVICT, RATELIMIT, TX_CACHE, and TX_DISK. SmartNIC does not maintain the state of the connections, so we eliminate the overhead for connection state synchronization between the SmartNIC and the host. Each core of SmartNIC performs the same simple operation, i.e., executing commands from the host.

OFFLOAD command is sent in a separate control packet that contains the file path and name of an object to offload to the SmartNIC cache, along with other information, including *nic_memory_offset*, *size*, and *offload_sequence*. When receiving the OFFLOAD command, SmartNIC reads the file with the size of *size* and stores the read file at *nic_memory_offset*, then sends a reply packet to the host with *offload_sequence* to let the host know that it successfully processed the offloading. When evicting less popular objects, the host sends an EVICT command, and SmartNIC evicts the corresponding object from its cache.

We perform the rate limiting on the host and SmartNIC to reduce the possibility of packet drops due to excessive transmission during a short period of time. RATELIMIT command is for requesting the rate limit in the SmartNIC. The rate decided by the host is included in the command.

We have two commands for data transmission. One is for data transmission by using the cache (TX_CACHE), and the other is for data transmission by file read (TX_DISK). These commands are transferred as TCP packets with the command information as the payload. For TX_CACHE, the SmartNIC replaces the payload with the object in the cache memory, and for TX_DISK, it reads the requested file and replaces the payload with the read file. After the packet modification, the packet goes through TSO and checksum calculation and is sent out.

3.3 Hiding the disk read latency using asynchronous disk I/O

We optimize the disk read operations at the host and SmartNIC using *io_uring* to solve the shortcomings of Linux *sendfile()* and mTCP's file reads. Linux *sendfile()* maps file pages to a sendfile buffer and enqueues the buffer to the TCP socket buffer, which enables the zero-copy. However, when a file page does not exist in memory, it causes the blocking of I/O and huge performance degradation of event-driven content servers [2]. Since *sendfile()* is not implemented in mTCP, the mTCP application must read the file into the application buffer and then copy the data to the mTCP send buffer through an mTCP API, *mtcp_write()*. Another copy occurs from the mTCP send buffer to DPDK packet buffer. Therefore, two copies are necessary for packet transmission. To solve the inefficiency of Linux's *sendfile()* and mTCP, IO-TCP's file read for content servers, we modify mTCP and implement *flexserver_sendfile* with the principles of the asynchronous non-blocking I/O and zero-copy transmission.

Figure 2 shows the operation of FlexServer's asynchronous file read using *io_uring*. We added a disk read thread (*disk_read_thread*) in addition to the mTCP's packet processing thread. First, when the application calls *flexserver_sendfile()* of the FlexServer Library (*libflexserver*), mTCP put the file read request into the file read request list (*frd_request_list*), and returns immediately without waiting for the completion. The disk read thread retrieves the request on the list and, after processing, submits it to the

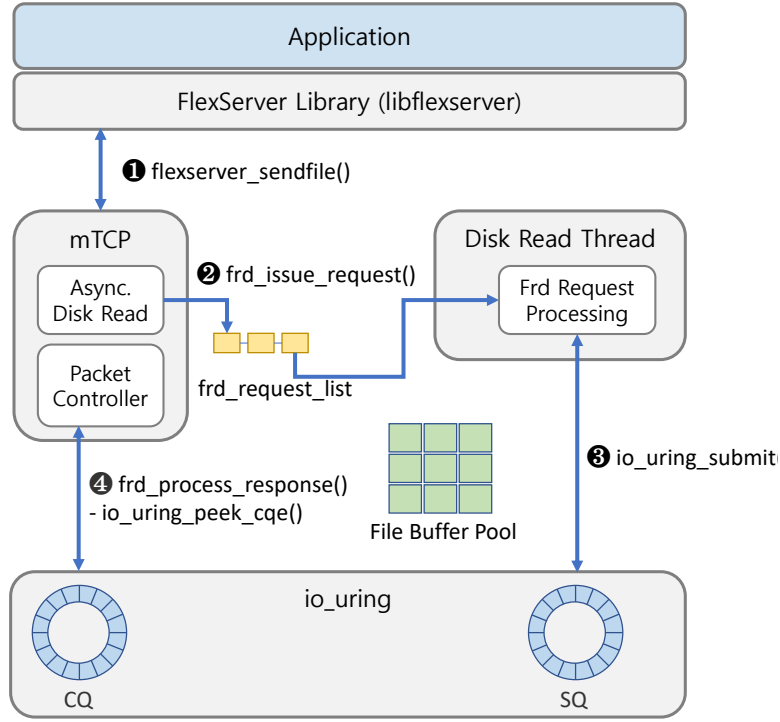


Figure 2: Operation of asynchronous file read using *io_uring*.

SQ. When the disk read is completed, *io_uring* put the completion information on the CQ. Then, mTCP retrieves the completion information and updates the sender buffer information of the flow that made the request. After that, when mTCP is processing the packets to be sent, it will send the packet for the flow. Since *flexserver_sendfile()* returns immediately without waiting for the read completion, the application can process other requests without the head of the line blocking problem.

In the mTCP, the application thread and the packet processing thread compete for the core. In our system, since we added the disk read thread, 3 threads will compete for the core. To guarantee the disk read processing throughput, the disk read thread retrieves the requests on the file read request list when the list size exceeds a threshold (16 in our system) or the timer expires (50 us in our system), sends to the *io_uring* SQ. While submitting the request, it passes the TCB of the flow so that when the disk read finishes, the mTPC can immediately find the flow information.

We eliminate the use of the file buffer cache since it incurs overhead whenever the new file is read. Instead we maintain cache on the host memory and SmartNIC memory. To disable the file buffer cache, we use `O_DIRECT` and `O_SYNC` flags when we open the file, and this requires an aligned memory address. Therefore, we maintain a fixed-sized file buffer pool with the aligned addresses. The file buffer is assigned to the flow when the file read is requested and freed when the file transmission is completed and TCP acknowledgment is received. The file buffer is processed in a similar way as the meta-data as explained in Section?? by using `data_len`, `head_seq`, `already_sent` fields.

Since the disk read completion rate varies, if we send the packet whenever the disk read completes, sometimes too many packets can be transferred, and significant packet drops can occur. To solve this

issue, we introduce rate-limiting for the packet transmission. We maintain an array to store the amount of bytes recently sent for all flows. When we retrieve the completion queue entry, we check whether sending the packet with the corresponding data will cause the throughput to be higher than the assigned throughput limit. If so, we delay the transmission and check the completion of other flows. The method to decide the throughput limit is described in Section 3.6.

For further performance optimizations, we use Scatter Gather DMA (SG-DMA) for packet transmission. DPDK supports the external buffer that abstracts the SG-DMA which enables the DMA of multiple non-contiguous memory regions. We set the IOVA for SG-DMA to point to the file buffers, and we can remove the use of memory copy for packet transmission. We also added TSO to the mTCP.

3.4 Hierarchical caching using the host and SmartNIC memory

FlexServer introduces a hierarchical caching mechanism using an L1 cache at the SmartNIC and an L2 cache at the host for popular objects. Each cache is composed of multiple blocks of the same size. One object can fit into a single block but may fit into multiple blocks. This cache design may waste memory due to objects that do not fit the block size. However, we trade the memory inefficiency for high performance. In order to manage the cache optimally, we devise an algorithm to decide which file objects to save in the SmartNIC cache and host cache.

Our caching algorithm operates between L1 and L2 cache and between L2 and disk. The algorithm runs every R request. It first selects *promotion candidates* from the lower level (L2 in the case of caching decision between L1 and L2, and disk in the case of caching decision between L2 and disk) through random sampling of M objects. It then calculates their priorities and selects the object with the highest priority as in [15] for the final object to cache. Let n_i , s_i , t_i , and p_i be the number of requests, the size, and the time from the cached time (or evict time in the case of objects in disk), and the priority of segment i , respectively. Let $\omega \in (0, 1)$ be a balancing parameter. Then, the priority is obtained by the following equation.

$$p_i \leftarrow (1 - \omega) \frac{n_i}{s_i t_i} + \omega p_i \quad (1)$$

Then, objects of the higher level cache are sampled as *demotion candidates* using the same sampling and priority calculation. After that, one object with the lowest priority is chosen as the final object demotion. We then switch two objects. If the object for promotion has a larger size in blocks, we choose multiple demotion objects.

To store an object in the SmartNIC cache (L1), we use an OFFLOAD command as explained in Section 3.2. To prevent any object from being accessed while the demotion from SmartNIC is processed, we propose a flow-segment binding inspired by [16]. We add two variables for the object: *itemActive* and *itemRefCount*. The *itemActive* is set to 0 the moment the object is designated as a demotion object, and no new flows can access that object. *itemRefCount* is incremented atomically each time a flow references an object in the cache and decremented atomically when a flow completes transferring the object. If an object is designated as a demotion object, we delay it until *itemRefCount* reaches zero. Also, when promoting an object, if the reply packet from the SmartNIC is received, *itemActive* is set to

1, and flows can access the promoted object. L2 Cache works in a similar way using the same variables.

3.5 Dual Buffer-based Consistency Management in the Control Plane

We aim for a separation of the control and data planes. For this, we propose a mechanism where the application header and network protocol headers are handled by the host's control plane, and the data processing is performed at the host or SmartNIC data plane. The data plane is informed about the location of the object in the cache or file information to read from disks to fill the packet received from the control plane with the retrieved object and transmit the final reconstructed packet. The advantages of this mechanism are that the data plane does not have state information about the flows, simplifying the overall system structure and operations and that the host can implement it without major modifications to the application and kernel TCP code.

The main issue here is that the meta-information about the object to be inserted into the intermediate packet passed to the data plane has a different size than the actual data. This size difference can break TCP consistency without meticulous processing. To solve this issue, we modify the TCP send buffer to have a dualized structure: *general send buffer* and *meta-data send buffer*. The *general send buffer* is responsible for the general TCP payload, such as the header of the application, and the *meta-data send buffer* is a linked list that manages the meta-information for the objects to be transferred. The meta-data contains the size of the object (*size*), its location in cache or file (*memory_offset*), the TCP sequence number for the object (*seq*), and the length of the successfully transmitted part (*already_sent*). The *memory_offset* is the location within the cache with multiple blocks, the *seq* is determined by considering the length of the general payload and the objects' sizes and their transmission order, and the *already_sent* is initially set to zero.

We add a *shape* header in front of the application header and object meta-data, which contains information about the number of general application data and object meta-data and their order. Our scheme can handle the case where multiple headers and objects are transferred in one stream, e.g., multiplexing in HTTP/2.

The operation of intermediate packet creation by using the dualized buffers in TCP is shown in Algorithm 1. When sending data, TCP assigns a sequence number matched with each byte. We set the current sequence number (*seq_cur*) to the byte to send next (*snd.next*) determined by the TCP protocol, and the size of the total payload to send (*to_send*) is determined by the TCP congestion window. Before inserting the payload into the packet, a *shape* containing the overall packet structure is added (*AddEntryShape()*). *shape* has fields containing the total number of meta-data and general data blocks (*n_blocks*) and a bit sequence of information about whether each block is general data or meta-data (*order*), as shown in Figure 3. If the current sequence number is less than the sequence number of the current metadata ($seq_cur < m.seq$), general data is used as the payload, and its size is determined as the smaller of the difference between the metadata and the current sequence number ($snd = m.seq - seq_cur$) and the remaining transmission amount (*to_send*). If the current sequence number is equal to or greater than the sequence number of meta-data ($seq_cur \geq m.seq$), it means that meta-data should be sent as pay-

ALGORITHM 1

Algorithm for creating an intermediate packet in the host TCP utilizing dual buffers.

```
1:  $seq\_cur \leftarrow snd\_nxt$  : Current TCP sequence number
2:  $to\_send$  : Remaining bytes in the congestion window
3:  $meta\_data\_buffer$  : Meta-data send buffer
4:  $m \leftarrow meta\_data\_buffer.head$ 
5:  $AddEntryShape(b, m)$ 
6: while  $to\_send > 0$  do
7:   if  $seq\_cur < m.seq$  then
8:      $snd \leftarrow m.seq - seq\_cur$ 
9:      $snd \leftarrow \min(snd, to\_send)$ 
10:     $CopyRealPayload(b, snd)$ 
11:   else
12:      $s \leftarrow (seq\_cur - m.seq)$ 
13:      $t \leftarrow m.size - s$ 
14:      $snd \leftarrow \min(to\_send, t)$ 
15:      $start \leftarrow m.memory\_offset + s$ 
16:      $AddMetaData(snd, start)$ 
17:     if  $snd > m.size - s$  then
18:        $m \leftarrow m.next$ 
19:     end if
20:   end if
21:    $to\_send \leftarrow to\_send - snd$ 
22:    $seq\_cur \leftarrow seq\_cur + snd$ 
23: end while
```

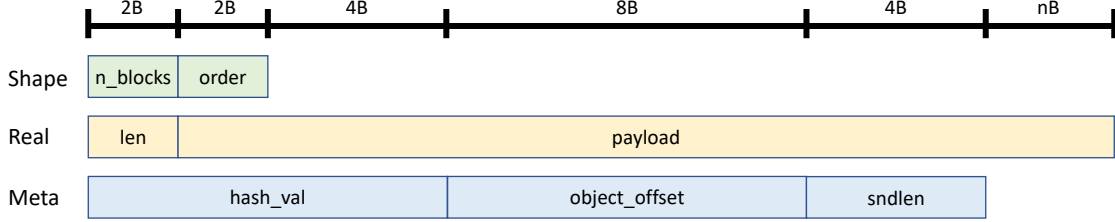
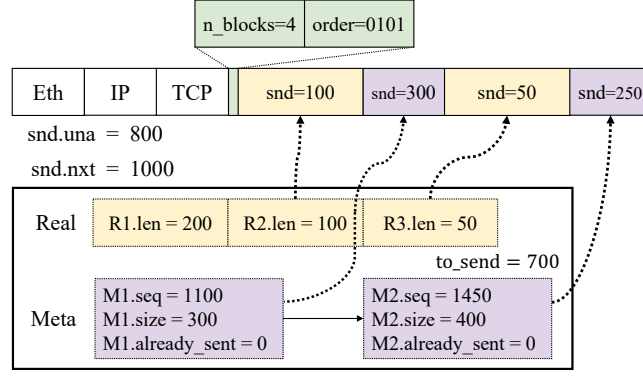


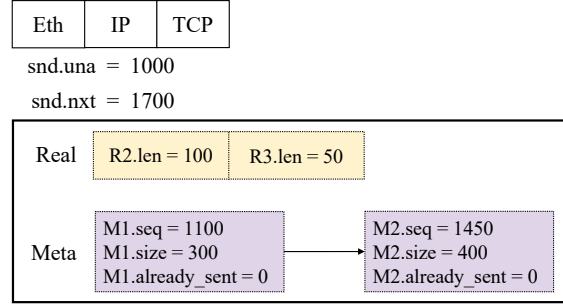
Figure 3: Structure of the fields in the intermediate packet.

load. In this case, considering the current sequence number (seq_cur), remaining transmission amount (to_send), meta-data size ($m.size$), etc., the amount of data to be transmitted (snd) and the starting address of the data in the cache or the file ($start$) are calculated. Then, the meta-data is inserted into the packet. When all the data represented by the metadata has been sent ($snd > m.size - s$), we move to the next metadata ($m \leftarrow m.next$). Then, we adjust the remaining send amount (to_send) and the current sequence number (seq_cur) to match the amount sent and repeat this procedure until the remaining transmission amount reaches zero.

Figure 4a is an example of creating an intermediate packet in the host TCP utilizing dual buffers. In this example, the application sends general data of 200 bytes, another general data of 100 bytes, a cached object of 300 bytes, another general data of 50 bytes, and another file object of 250 bytes. Suppose the remaining bytes in the congestion window (to_send) is 700, the sequence number for the last acknowledged byte (seq_ack) is 800, and the current sequence number (seq_cur) is 1000. Note that



(a) Sending a TCP data packet.



(b) Receiving an ACK packet.

Figure 4: Example of the host TCP operations utilizing dual buffers.

we assume we already sent the first real data of 200 bytes but did not receive ACK packets for these bytes.

Since the current sequence number is larger than the first meta-data sequence number ($seq_{cur} > M1.seq$), we know that the general data should be at the front. And, since the current sequence number is larger than the last ACKed byte by 200, we first insert $R2$, and the current sequence number is updated as $seq_{cur} = 1000 + 100 = 1100$. Also, to_send is decremented by 100 to become 600. Then, since the current sequence number matches the first meta-data sequence number ($M1.seq$) and the remaining transmission amount is larger than the size of the data for the first meta-data ($to_send > P1.size$), we insert a meta-data for $M1$ into the packet. Then, the remaining transmission amount (to_send) and the current sequence number (seq_{cur}) are decremented/incremented by the size of data for the first meta-data ($P1.size$). Using the same method, we put the third general data ($R3$) in front of the second meta-data ($M2$). When processing the second meta-data ($M2$), we need to instruct the data plane to send only the first 250 bytes because the remaining transmission amount (to_send) becomes 250. The rest of the processing is the same as the first meta-data ($M1$).

If the packet arrives at the client successfully, the client's TCP will send an ACK packet. According to the received ACK, the TCP should remove the appropriate general data or meta-data from the dual buffer. The algorithm for removing the data in the general send buffer and meta-data send buffer in TCP is shown in Algorithm 2.

Figure 4b depicts the situation when the client receives an ACK with an acknowledgment number

ALGORITHM 2

Algorithm for removing the data in the general send buffer and meta-data send buffer in TCP.

```
1:  $b$  : Dual buffer
2:  $b.head\_seq$  : Dual buffer head sequence number
3:  $b.meta$  : Head meta-data
4:  $to\_rm \leftarrow ack - b.head\_seq$ 
5: while  $to\_rm > 0$  do
6:   if  $b.head\_seq < b.meta.seq$  then
7:      $rm \leftarrow b.meta.seq - b.head\_seq$ 
8:      $rm \leftarrow \min(rm, to\_rm)$ 
9:      $RemoveGeneralPayload(b, rm)$ 
10:     $Incr(b.head\_seq, rm)$ 
11:     $to\_rm \leftarrow to\_rm - rm$ 
12:   else
13:      $remain \leftarrow b.meta.size - b.meta.already\_sent$ 
14:     if  $to\_rm < remain$  then
15:        $Incr(b.meta.already\_sent, to\_rm)$ 
16:        $Incr(b.head\_seq, to\_rm)$ 
17:        $to\_rm \leftarrow 0$ 
18:     else
19:        $rm \leftarrow \min(remain, to\_rm)$ 
20:        $RemoveHeadMeta(b)$ 
21:        $Incr(b.head\_seq, rm)$ 
22:        $to\_rm \leftarrow to\_rm - rm$ 
23:     end if
24:   end if
25: end while
```

of 1700 after sending a data packet as in Figure 4a. We assume $R1$ has been freed from the buffer already. Considering the acknowledgment number ($seg.ack$) and oldest unacknowledged sequence number ($snd.una$), we know that the data length that needs to be removed from the buffer is $to_rm = 700$. First, by comparing $snd.una$ with $M1.seq$, we know that $R2$ is the payload before $M1$. Therefore, we remove $R2.len$ from the buffer and get $to_rm = 600$ and $snd.una = 1100$. We know that $snd.una$ and $M1.seq$ are the same and $M1.size < to_rm$, so we remove all of $M1$. In this way, we can continue the buffer elimination. Meanwhile, just before we remove $M2$, we see that $to_rm = 250$. In this case, instead of removing all of $P2$, we increment $M2.already_sent$ by 250. We remove $M2$ when we receive an ACK with an acknowledgment number greater than or equal to 1850.

3.6 Intelligent load balancing between the host and SmartNIC

We devise a load-balancing mechanism to assign three types of content, normal data, file data, and cached data, to the host and SmartNIC. The normal data is processed in the same location, i.e., host or SmartNIC, with the adjacent data. For example, the HTTP header is processed in the host if the file in the body is processed in the host. The decision for cached data is simple. If the object is in the L1 cache, it is sent to the SmartNIC, while if the object is in the L2 cache, it is sent to the host. The remaining decision is where to process the file data. Since our objective for load-balancing is to assign as much workload as possible to SmartNIC, we have to find SmartNIC's file data transmission capacity. It depends on

how much traffic is sent through the L1 cache because the resource consumption for cached data and file data are different. We obtain the file data transmission capacity empirically. As shown in Figure 9, we measure the maximum throughput by SmartNIC file transmission while varying the throughput by SmartNIC cache transmission. We use this result to decide the number of requests to send to SmartNIC for file transmission. The SmartNIC periodically measures the throughput by cache and reports to the host. Then, the host finds the remaining file transmission capacity of SmartNIC, and when it decides where to send the request for file transmission, it chooses SmartNIC if the recent average throughput of file transmission at SmartNIC is less than the capacity. If it is larger than the capacity, it chooses the host.

IV Implementation

4.1 Host

We implement FlexServer on a server with the Intel Xeon Silver 4310 CPU and 96GB of DDR4 memory. We install the memory so that we can utilize the multiple memory channels. The OS is Ubuntu 20.04.6 LTS, and the Kernel version is 5.8.0. FlexServer library is implemented by 5453 LOC, and mTCP is modified by 6871 LOC. It provides *flexserver_cache_send()*, *flexserver_sendfile()* APIs. *flexserver_cache_send()* performs object transfer through the L1 cache and L2 cache depending on the current cache status and returns the length of the transferred object in case of success or an error value in case of failure. Meanwhile, *flexserver_sendfile()* reads data from disk in an asynchronous/non-blocking method in the host or SmartNIC, depending on the SmartNIC status, and transfers it. It returns the length of the file if it was successful in a disk read and packet transfer. Otherwise, it returns an error value.

4.2 SmartNIC

The NIC stack is implemented using DPDK with 4954 LOC. Packets from the host are distributed to specific threads bound to specific cores via RSS using 4 tuples. The NIC stack handles two offloads: file read and L1 cache hit. We implement a memory pool for the L1 cache and a hash table to find objects. Since the cached objects are shared by all flows, the cache memory pool and the hash table are shared by all threads. When sending the packet on the wire, we utilize DMA for separate memory spaces using SG-DMA. For file reads, each thread manages its own aligned memory buffer pool. When an offloading command for a specific file read comes from the host, it performs a file read through the *io_uring* API. Similar to the L1 cache object transfer, it utilizes SG-DMA to transfer packets to the wire.

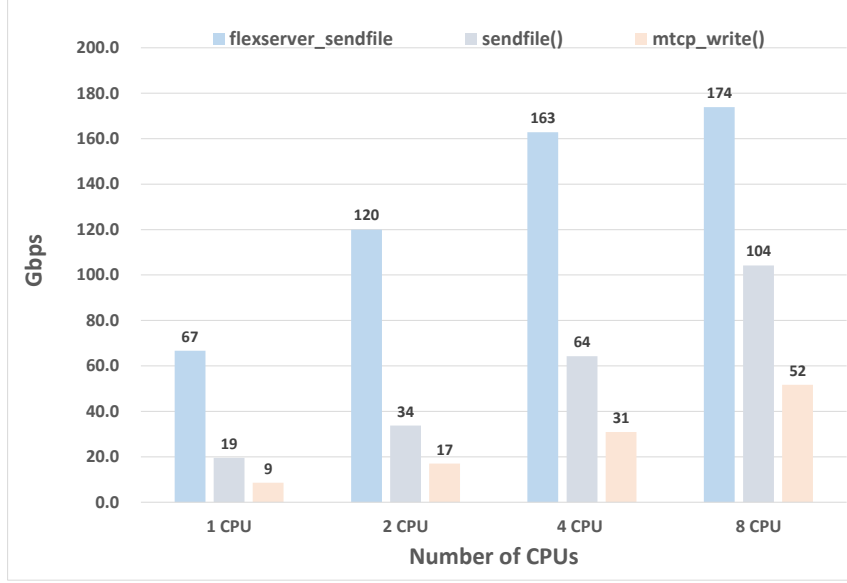


Figure 5: Throughput of FlexServer, Linux with *sendfile()*, and mTCP when disk reads are performed on the host while varying the number of CPU cores.

V Evaluation

In this Section, we evaluate FlexServer by running experiments. We ask the following questions for the experiments. (1) how much performance benefit does FlexServer provide by its mechanism of asynchronous I/O? (2) is FlexServer scalable in terms of the number of CPU cores? (3) how much throughput can FlexServer achieve at the SmartNIC through the caching and file read offloading?

5.1 Experiment Setup

Our testbed for the experiments consists of two machines: one server equipped with a SmartNIC and one client. We run the *lighttpd* [17] web server application on the server by porting it to use mTCP. Then, we create HTTP content requests using the client through the HTTP stress tool, *wrk* [18]. *Wrk* provides the request generations following a custom content popularity distribution through Lua scripts. However, the process of executing Lua scripts consumes a significant amount of CPU cycles on the client machine. Therefore, we modified *wrk* to generate Zipf-distribution requests without using Lua scripts. HTTP requests are known to follow a Zipf-like distribution [19]. We use 4 NVMe SSDs to store content for the experiments.

5.2 Experimental Results

The first experiment tests the performance of FlexServer’s *io_uring*-based I/O operations compared to the Linux and mTCP. We let FlexServer send files via *flexserver_sendfile()*, while we use *sendfile()* for Linux. For mTCP, we use *mtcp_write()*. The experiment was run with increasing CPU cores from 1 to 8. The total content size is 200GB of 512KB objects on the server side. The total number of objects is

Table 1: CPU cycle consumption comparison of FlexServer, Linux, and mTCP.

	Network Processing	Application Processing	File IO System Calls	Memory Copy	etc
FlexServer	43.8	28.23	7.27	0	20.7
Linux	14.56	14.2	49.73	0	21.51
mTCP	51.9	5.22	24.17	16.86	1.85

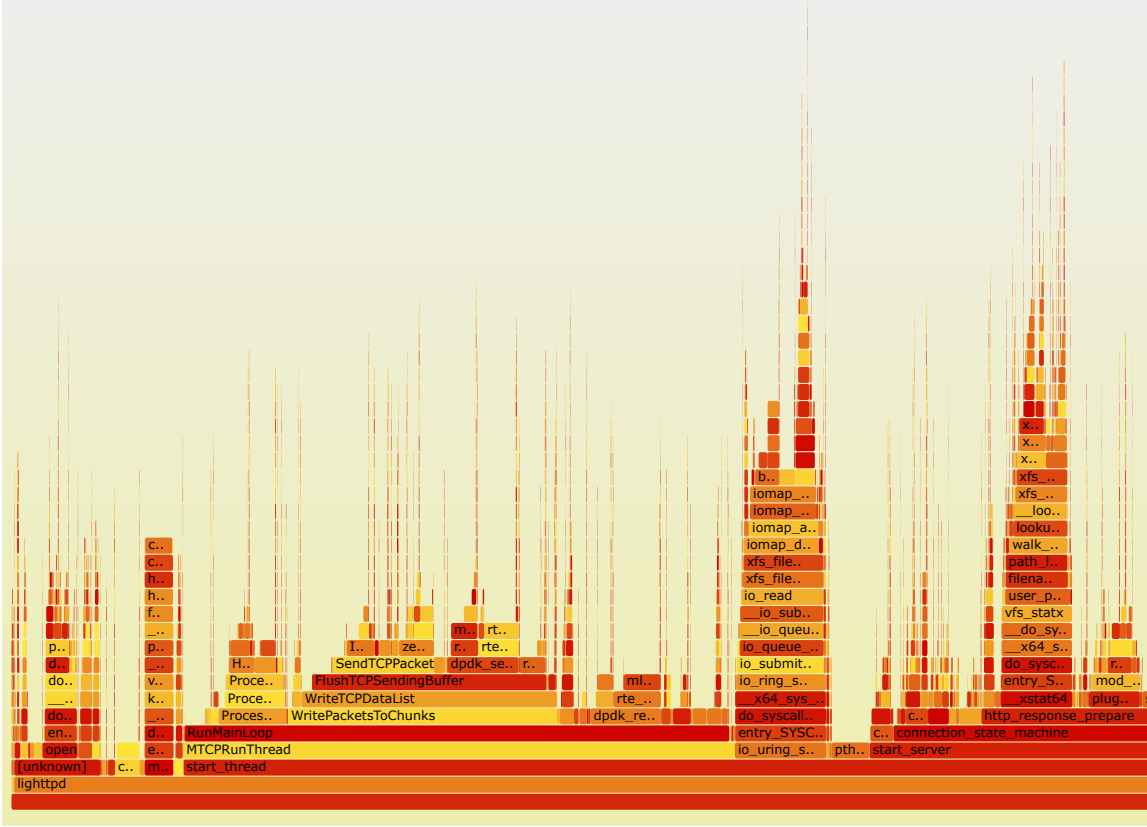


Figure 6: Flame graph when running FlexServer with 1 CPU core.

409,600. The client application repeatedly requests one of the objects on the server side with uniform probability. We run 30 threads on the wrk tool.

Figure 5 shows the experiment results. *flexserver_sendfile()* achieves 67 Gbps with only 1 CPU core and reaches 174 Gbps with 8 CPU cores. In contrast, Linux’s *sendfile()* achieves 104 Gbps with 8 CPU cores, and mTCP reaches 52 Gbps with 8 CPU cores. *sendfile()* has the characteristic of blocking I/O when the file page does not exist in memory. Since, in this experiment, HTTP requests are generated according to a uniform distribution, there is a high chance that the file page does not exist on the system. For *mtcp_write*, the performance is very low compared to *flexserver_sendfile()*. Data transfer using *mtcp_write()* first reads data into the application buffer, then the application copies the data to the mTCP send buffer, and then copies the data again to the DPDK packet buffer. The performance of mTCP is poor because the application is blocked until the file read is complete, and there are many unnecessary copies. When analyzed through Perf, the file IO system call takes up 24.17% of the total cycles, and the memory copy takes up 16.86%. *flexserver_sendfile()* solves the shortcomings of mTCP by receiving file IO requests, reading files asynchronously into a file buffer via *io_uring*, and then sending them

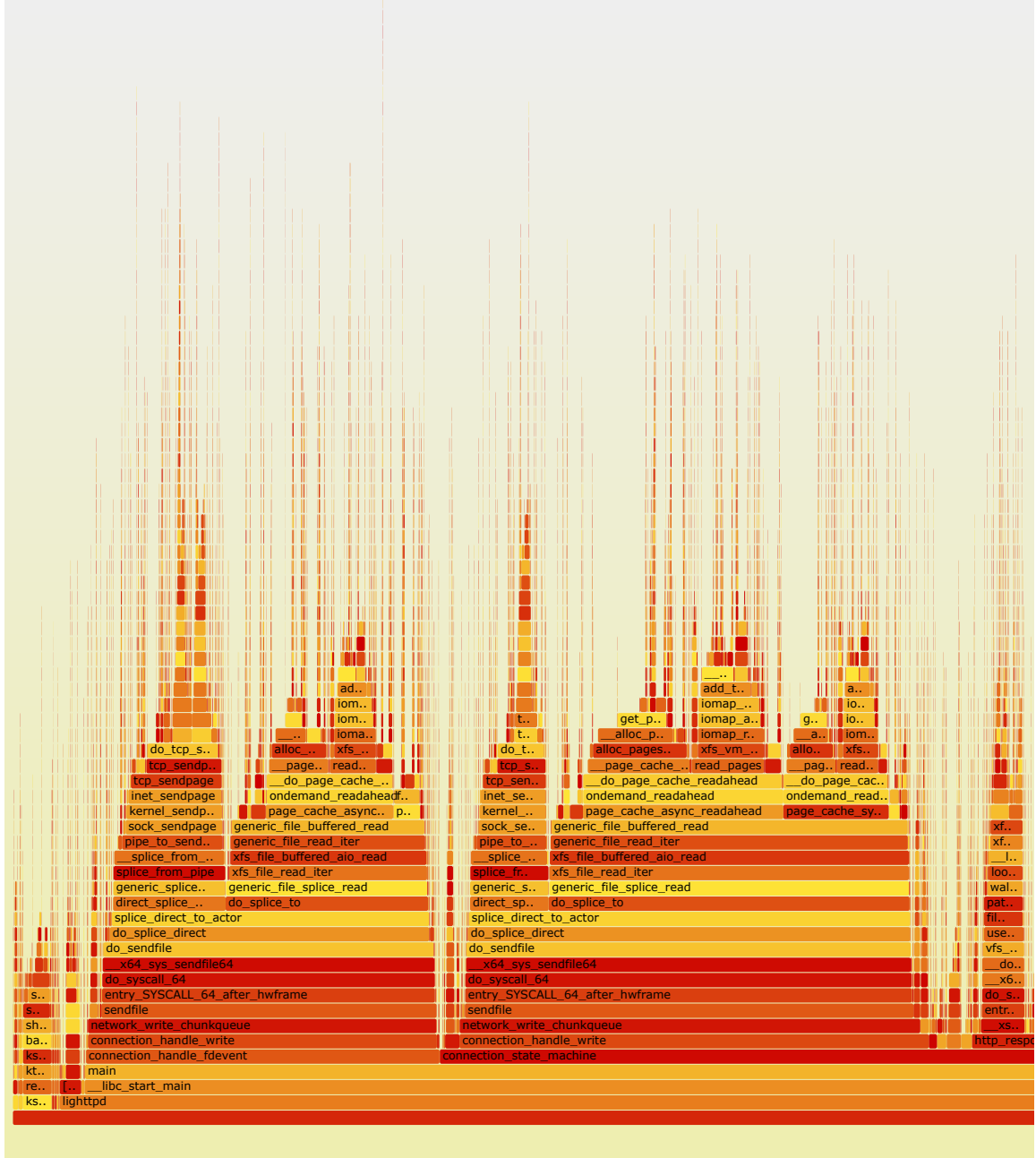


Figure 7: Flame graph when running Linux with 1 CPU core.

externally via SG-DMA. When analyzing the results through Perf, the overhead due to memory copy becomes zero, and the system call due to the file IO took up 7.27% of the total CPU cycles. As a result, the application received enough cycles, which led to performance improvement. The CPU cycle consumption of FlexServer, Linux, and TCP is compared in Table 1. The detailed CPU consumption of FlexServer, Linux, and TCP is shown in Figure 6, Figure 7, and Figure 8, respectively.

To find how much throughput FlexServer achieves at the SmartNIC through the caching and file read offloading, we run another experiment. We first set the L1 cache size to 2GB and then fill the L1 cache with 512KB of objects. We then run the experiment with a total of 800 flows, varying the ratio of

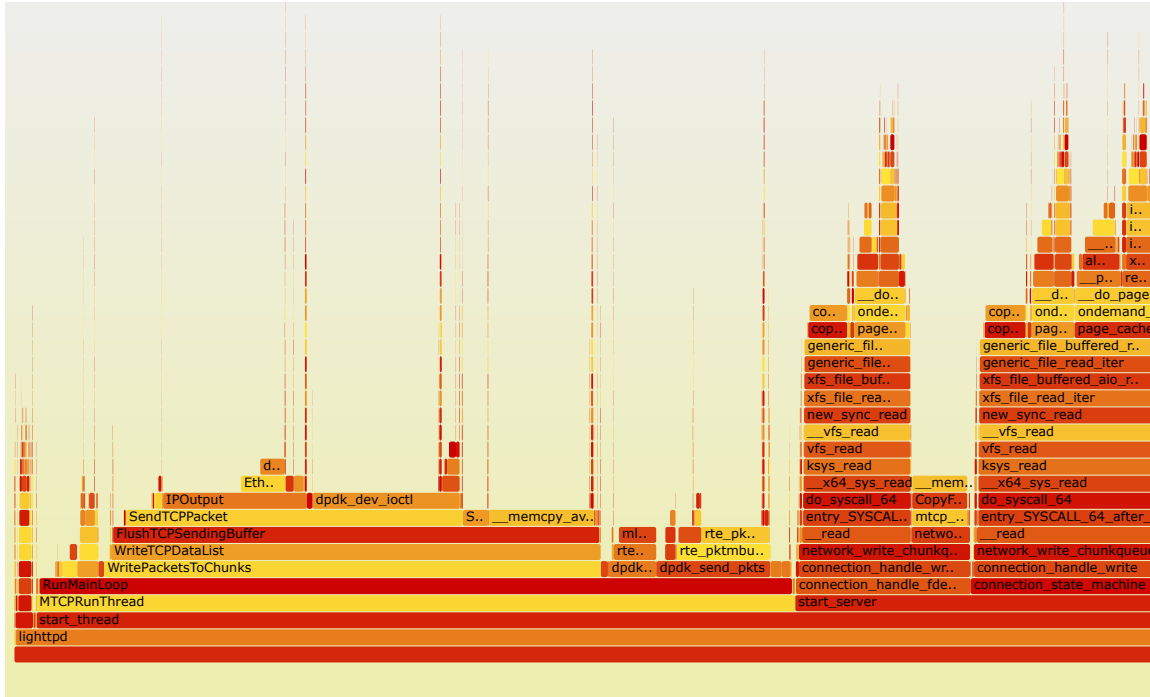


Figure 8: Flame graph when running mTCP with 1 CPU core.

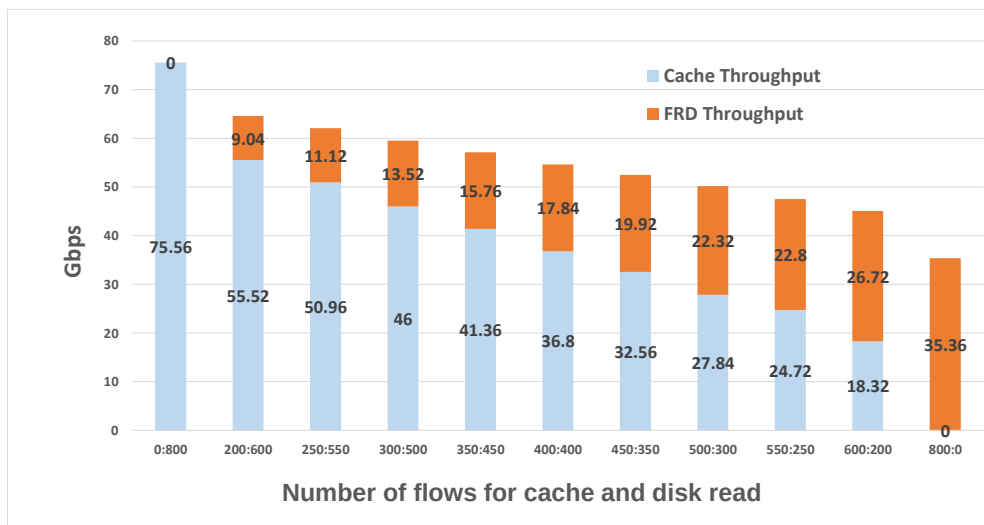


Figure 9: Throughput by SmartNIC cache and file reads when varying the number of cache hit flows and file read flows.

flows between those requesting only objects in the L1 cache and those transferring data directly from the SmartNIC via file reads. The maximum throughput reaches 75.56 Gbps when all the flows are served by the cache. The throughput performance increases with a higher number of flows requesting objects that hit the L1 cache. While the number of disk reads is a contributing factor, the main factor seems to be the low memory bandwidth of the SmartNIC. When the SmartNIC needs to serve the objects by directly

accessing the disks, it retrieves the data through P2P DMA, which consumes memory bandwidth. This is likely to be overcome when the memory bandwidth of the SmartNIC increases, e.g., Bluefield 3 DPUs. We use the data from this experiment to obtain the relationship between the caching and disk read offloading and their throughputs when they co-exist through a polynomial curve fitting and use it to decide how many disk read requests will be offloaded to SmartNIC.

VI Related Work

SmartNIC offloading. SmartNICs have been used in a variety of applications to offload tasks that would otherwise be handled by the host CPU. iPipe [8] is an actor-based offloading framework for distributed applications, where each actor has a private state and communicates with other actors through message exchanges. λ -NIC [9] is for offloading lambdas, small, fine-grained custom programs, to smartNICs in a serverless computing environment, allowing thousands of lambdas to run on a single NIC with tail-latency guarantees. KV-Direct [10] is a key-value system that uses FPGA-based SmartNICs to extend RDMA primitives to provide direct access to the host’s main memory for key-value operations. SmartNICs such as [11,12] support the acceleration of mobile access and core networks. AccelTCP [13], FlexTOE [14], and IO-TCP [4] are techniques for accelerating TCP using SmartNICs. AccelTCP [13] offloads complex TCP operations such as connection setup and teardown to the NIC and allows packet relay between spliced connections to operate on the SmartNIC. FlexTOE [14] is another approach for TCP offloading, which decomposes the TCP data path into a data-parallel pipeline of processing modules using NPU SmartNICs. IO-TCP [4] is a technique for offloading disk I/O and TCP packet transfers to SmartNICs to increase the performance of online content delivery. DASHER differs from these SmartNIC-based TCP offloading techniques in that it separates the control plane and the data plane in the DASH environment, increasing the efficiency of the data plane by storing popular video segments on SmartNICs and accessing them at the end of the data plane.

VII Conclusion

In this thesis, we presented FlexServer, the first high-performance content server that supports a line rate of 200 Gbps NICs. We use a SmartNIC to offload disk read and packet transfer operations. To reduce the overhead of disk read operations, we utilize the up-to-date asynchronous I/O mechanism, *io_uring*, and hide the latency of disk read operations. To reduce the inefficiency of duplicate transmission of popular content, we introduce a caching mechanism both at the server and the SmartNIC. We also develop a mechanism to allocate the workload to the host and SmartNIC dynamically. We deploy a dual buffer-based consistency management algorithm to the control plane for complete consistency with TCP. We implement FlexServer using the mTCP user-level TCP stack and an NVIDIA Bluefield 2 SmartNIC with a 200 Gbps Ethernet port. Via evaluations, we show that FlexServer can achieve the line rate of 200 Gbps with only a single host CPU core in typical workload scenarios.

References

- [1] “2023 global internet phenomena report,” <https://www.sandvine.com/global-internet-phenomena-report-2023>.
- [2] I. Marinos, R. N. Watson, M. Handley, and R. R. Stewart, “Diskl cryptl net: rethinking the stack for high-performance video streaming,” in *Proceedings of the Conference of the ACM Special Interest Group on Data Communication*, 2017, pp. 211–224.
- [3] “Intel data direct i/o technology,” <https://www.intel.com/content/www/us/en/io/data-direct-i-o-technology.html>.
- [4] T. Kim, D. M. Ng, J. Gong, Y. Kwon, M. Yu, and K. Park, “Rearchitecting the {TCP} stack for {I/O-Offloaded} content delivery,” in *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*, 2023, pp. 275–292.
- [5] “io_uring,” https://man.archlinux.org/man/io_uring.7.en.
- [6] D. Didona, J. Pfefferle, N. Ioannou, B. Metzler, and A. Trivedi, “Understanding modern storage apis: a systematic study of libaio, spdk, and io_uring,” in *Proceedings of the 15th ACM International Conference on Systems and Storage*, 2022, pp. 120–127.
- [7] Z. Ren and A. Trivedi, “Performance characterization of modern storage stacks: Posix i/o, libaio, spdk, and io_uring,” in *Proceedings of the 3rd Workshop on Challenges and Opportunities of Efficient and Performant Storage Systems*, 2023, pp. 35–45.
- [8] M. Liu, T. Cui, H. Schuh, A. Krishnamurthy, S. Peter, and K. Gupta, “Offloading distributed applications onto smartnics using ipipe,” in *Proceedings of the ACM Special Interest Group on Data Communication*, 2019, pp. 318–333.
- [9] S. Choi, M. Shahbaz, B. Prabhakar, and M. Rosenblum, “ λ -nic: Interactive serverless compute on programmable smartnics,” in *2020 IEEE 40th International Conference on Distributed Computing Systems (ICDCS)*. IEEE, 2020, pp. 67–77.
- [10] B. Li, Z. Ruan, W. Xiao, Y. Lu, Y. Xiong, A. Putnam, E. Chen, and L. Zhang, “Kv-direct: High-performance in-memory key-value store with programmable nic,” in *Proceedings of the 26th Symposium on Operating Systems Principles*, 2017, pp. 137–152.

- [11] “N5013/n5014 fpga smartnic,” <https://www.silicom-usa.com/pr/4g-5g-products/4g-5g-adapters/n5013-n5014-fpga-smartnic-intel-stratix-based/>.
- [12] “Xilinx 5g telco accelerator cards,” <https://www.xilinx.com/publications/presentations/xilinx-5g-telco-accelerator-cards.pdf>.
- [13] Y. Moon, S. Lee, M. A. Jamshed, and K. Park, “{AccelTCP}: Accelerating network applications with stateful {TCP} offloading,” in *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)*, 2020, pp. 77–92.
- [14] R. Shashidhara, T. Stamler, A. Kaufmann, and S. Peter, “{FlexTOE}: Flexible {TCP} offload with {Fine-Grained} parallelism,” in *19th USENIX Symposium on Networked Systems Design and Implementation (NSDI 22)*, 2022, pp. 87–102.
- [15] A. Blankstein, S. Sen, and M. J. Freedman, “Hyperbolic caching: Flexible caching for web applications,” in *USENIX Annual Technical Conference*, 2017.
- [16] H. Lim, D. Han, D. G. Andersen, and M. Kaminsky, “{MICA}: A holistic approach to fast in-memory key-value storage,” in *11th {USENIX} Symposium on Networked Systems Design and Implementation ({NSDI} 14)*, 2014, pp. 429–444.
- [17] “Lighttpd - fly light,” <https://www.lighttpd.net/>.
- [18] “wrk - a http benchmarking tool,” <https://github.com/wg/wrk>.
- [19] L. Breslau, P. Cao, L. Fan, G. Phillips, and S. Shenker, “Web caching and zipf-like distributions: Evidence and implications,” in *IEEE INFOCOM’99. Conference on Computer Communications. Proceedings. Eighteenth Annual Joint Conference of the IEEE Computer and Communications Societies. The Future is Now (Cat. No. 99CH36320)*, vol. 1. IEEE, 1999, pp. 126–134.

Acknowledgements

First of all, I would like to express my deep appreciation to my advisors, Prof. Youngbin Im, who provided motivation, encouragement, and guidance during the research and writing of the dissertation. Also, I sincerely thank my colleagues in N2SL lab : Bongwon Lee, Seunghyck Choi, Hyusnsu Kim Daeun Lee.

Lastly, I would like to express my sincere gratitude to my parents and my younger broder for their love and suppport throughout my entire life.

